

Event-Driven Architecture

Testing systems that asynchronously produce and consume events.

Events

Producers publish events and consumers react without direct call coupling.

Official site and documentation

Use the official Event-Driven Architecture source as the primary reference:

<https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/event-driven>. Official documentation should be the first source for installation, supported versions, syntax changes and new features.

QA risks

Ordering, duplication, loss, delay, incompatible schema and partial processing are central risks.

Testing

Validate schema, idempotency, retry, DLQ, correlation and observable consumer effects.

Messaging

Kafka, queues and cloud services require strategy for data, observability and cleanup.

When to use it

Use this knowledge when product risk, architecture or the delivery stage requires this type of validation. Confirm the goal, environment, required data and expected evidence before starting.

How QA applies it

Start from the behavior or risk to validate, prepare data and environment, execute the technique or tool reproducibly and capture evidence that helps the team understand the result.

What it is and where it fits

Event-Driven Architecture is a tool or technology that can support Quality Engineering work. Before adopting it, understand the problem it solves, where it fits in the workflow and what evidence or productivity gain it should provide.

How to start safely

Start by mapping producers, consumers, event schemas, brokers and delivery guarantees. Define correlation IDs, observability and deterministic test data before automating asynchronous flows, and validate retries,

duplication and ordering explicitly.

First QA-oriented use

Pick one small, verifiable task in Event-Driven Architecture, configure only what is necessary, run a simple example and confirm the outcome. Repeat with different data or conditions before adding abstractions.

Using it in a real project

Define where Event-Driven Architecture fits, who owns configuration, how credentials and data are handled and how results are shared. Integrate it into the team workflow instead of creating an isolated activity.

Adoption good practices

Standardize versions and configuration, document important decisions, keep secrets out of code and preserve reproducible examples. Reassess whether Event-Driven Architecture still delivers value as the product evolves.

Common issues and troubleshooting

When Event-Driven Architecture fails, verify versions, dependencies, permissions, connectivity, data and environment before assuming a product defect. Preserve the original error and reduce the issue to the smallest reproducible case.